behavior of the link. Each requires JavaScript to be enabled in the browser.

The `:method` option is a hack—it allows you to make the link look to the application as if the request were created by a `POST`, `PUT`, `PATCH`, or `DELETE`, rather than the normal `GET` method. This is done by creating a chunk of JavaScript that submits the request when the link is clicked—if JavaScript is disabled in the browser, a `GET` will be generated.

The `:data` parameter allows you to set custom data attributes. The most commonly used one is the `:confirm` option, which takes a short message. If present, an unobtrusive JavaScript driver will display the message and get the user's confirmation before the link is followed:

```
<%= link_to "Delete", product_path(@product),
                      method: :delete,
                      data: { confirm: 'Are you sure?' }
%>
```

The `button_to` method works the same as `link_to` but generates a button in a self-contained form rather than a straight hyperlink. This is the preferred method of linking to actions that have side effects. However, these buttons live in their own forms, which imposes a couple of restrictions: they cannot appear inline, and they cannot appear inside other forms.

Rails has conditional linking methods that generate hyperlinks if some condition is met or just return the link text otherwise. `link_to_if` and `link_to_unless` take a condition parameter, followed by the regular parameters to `link_to`. If the condition is `true` (for `link_to_if`) or `false` (for `link_to_unless`), a regular link will be created using the remaining parameters. If not, the name will be added as plain text (with no hyperlink).

The `link_to_unless_current` helper creates menus in sidebars where the current page name is shown as plain text and the other entries are hyperlinks:

```
<ul>
<% %w{ create list edit save logout }.each do |action| %>
   <li>
```